

Experiment No: 4

Aim: Hands on Solidity Programming Assignments for creating Smart Contracts

Theory:

1. Basic Solidity Concepts: Data Types, Variables and Functions

Solidity is a programming language specifically designed for developing smart contracts on blockchain platforms such as Ethereum. It provides several built-in data types to represent different kinds of information. Numerical values can be represented using `uint` and `int`, while `bool` is used for logical conditions. The `address` type represents an Ethereum account or contract address. Text and raw binary information can be handled using `string` and `bytes`.

Variables in a smart contract may have different scopes and lifetimes. **State variables** retain their values on the blockchain, whereas **local variables** are used temporarily while a function is running. Solidity also provides special global variables such as `msg.sender`, which identifies the caller, and `block.timestamp`, which gives the current block timestamp.

Functions define the operations that a smart contract can perform. A **view** function can access contract data without changing it, while a **pure** function performs calculations without accessing blockchain state.

2. Function Arguments and Returned Results

Functions use **parameters** to receive information from the caller. These values can then be processed according to the logic defined inside the function.

A function can provide its result using the **returns** keyword. Solidity supports returning multiple values from a single function, which is useful when several related results need to be obtained together. Return variables can also be named to make the code more understandable.

For example, a mathematical function can receive two integers and return their addition, subtraction, or multiplication.

3. Function Access, Modifiers and Constructors

Visibility determines the scope from which a function can be called. Solidity provides four main visibility levels: **public**, **private**, **internal**, and **external**. Public functions can be accessed from outside as well as inside the contract, whereas private functions are restricted to the contract itself. Internal functions can also be used by derived contracts. External functions are primarily designed for calls originating outside the contract.

A **modifier** provides a convenient way to place common conditions around functions. For example, a modifier can ensure that only an authorized account is allowed to execute a particular operation.

A **constructor** is a special function that executes automatically at the time of contract creation. It is mainly used to establish the initial configuration of the contract, such as assigning the deployer's address as the owner.

4. Conditional and Repetitive Execution

Control-flow statements determine the order in which instructions are executed within a smart contract.

The **if**, **else if**, and **else** statements allow the contract to choose an action based on whether a particular condition is satisfied. They are commonly used for validation and decision-making.

Solidity also supports **for**, **while**, and **do-while** loops for executing a group of statements repeatedly. However, loops should be designed carefully because blockchain operations consume gas. A loop that performs a large number of operations can require significant gas and may even make a transaction impractical.

5. Organizing Data Using Solidity Data Structures

Different data structures are available in Solidity to store and manage information efficiently.

An **array** contains a collection of elements arranged in a particular order. Arrays can either have a predetermined size or grow dynamically.

A **mapping** stores information using a key and its corresponding value. It is particularly useful for directly accessing information associated with an identifier, such as an Ethereum address and its balance.

A **struct** is a custom data type that combines multiple related fields into one logical unit. For example, information about a student can contain a name, roll number, and marks within a single structure.

An **enum** is useful when a variable should contain one value from a fixed set of possible options. It can therefore be used to represent stages or states of a contract.

6. Solidity Data Storage Locations

Solidity categorizes data into different locations according to how it is stored and used.

Storage is the permanent data area associated with a smart contract. Values stored here remain available between transactions and are recorded on the blockchain.

Memory is used for temporary information during the execution of a function. Once the function finishes, memory data is no longer retained.

Calldata is another temporary location that is primarily used for arguments of external functions. It is read-only, meaning that the function cannot modify the original calldata. It can also reduce unnecessary data copying and therefore help improve gas efficiency.

7. Ether, Wei, Gas and Blockchain Transactions

Ether (ETH) is the native currency used within the Ethereum network. For handling very small amounts, Ether is represented in **Wei**, which is its smallest commonly used denomination.

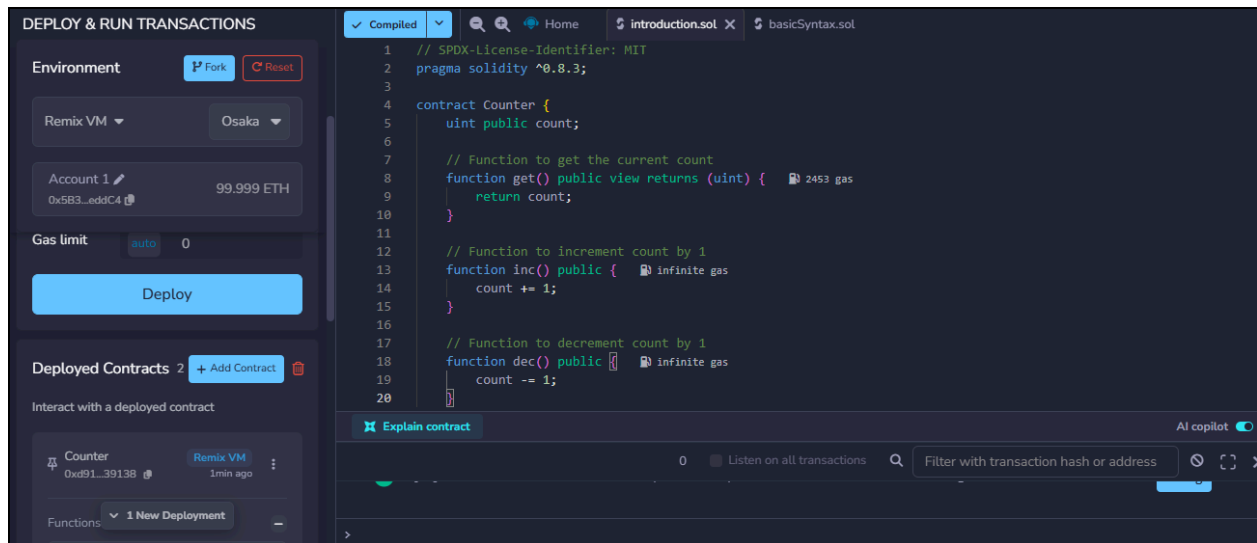
1 Ether = 10^{18} Wei

Smart contract execution requires computational resources, and these resources are measured using **gas**. The final transaction cost depends on the amount of gas consumed and the gas price associated with the transaction.

Transactions are responsible for carrying out actions on the blockchain, such as transferring Ether or calling functions that change contract data. Solidity provides different mechanisms for transferring Ether, including `transfer()`, `send()`, and `call()`. Selecting an appropriate method and minimizing unnecessary computation helps in creating more efficient smart contracts.

Output:

Tutorial 1.



Tutorial 2.

LEARNETH

Tutorials list

Syllabus

2. Basic Syntax

2 / 19

state variables. We will look into them in the following sections.
To help you understand the code, we will link in all following sections to video tutorials from the [creator](#) of the Solidity by Example contracts.
[Watch a video tutorial on Basic Syntax.](#)

★ Assignment

- Delete the HelloWorld contract and its content.
- Create a new contract named "MyContract".
- The contract should have a public state variable called "name" of the type string.
- Assign the value "Alice" to your new variable.

Check Answer

Show answer

Next

Well done! No errors.

Compile

Home

introduction.sol

basicSyntax.sol

Create a DApp with your contracts Start now

```
1 //SPDX-License-Identifier: MIT
2
3 // compiler version must be greater than or equal to 0.8.3 and less than 0.9.0
4 pragma solidity ^0.8.3;
5
6 contract MyContract {
7     string public name = "Alice";
8 }
```

Explain contract

AI copilot

0 Listen on all transactions Filter with transaction hash or address

logs: 0 hash: 0x811...f421b

Tutorial 3.

REMUX 2.5.6

Learnth Solidity Beginner Course

Link Sign In See Plans Get AI Credits

LEARNETH

Tutorials list

Syllabus

3. Primitive Data Types

3 / 19

Watch a video tutorial on Primitive Data Types.

★ Assignment

- Create a new variable `newAddr` that is a `public address` and give it a value that is not the same as the available variable `addr`.
- Create a `public` variable called `neg` that is a negative number, decide upon the type.
- Create a new variable, `newU` that has the smallest `uint` size type and the smallest `uint` value and is `public`.

Check Answer

Show answer

Next

Well done! No errors.

Compile

Home

introduction.sol

basicSyntax.sol

primitiveDataTypes.sol

Create a DApp with your contracts Start now

```
19
20
21 Negative numbers are allowed for int types.
22 Like uint, different ranges are available from int8 to int256
23 */
24 int8 public i8 = -1;
25 int public i256 = 456;
26 int public i = -123; // int is same as int256
27
28 address public addr = 0xCA35b7d915458EF540aDe6068dFe2F44E8fa733c;
29 address public newAddr = 0x742d35Cc6634C0532925a3b844Bc454e4438f44e;
30 int public neg = -1;
31 uint public newU = 0;
32 // Default values
33 // Unassigned variables have a default value
34 bool public defaultBool; // false
35 uint public defaultUint; // 0
36 int public defaultInt; // 0
37 address public defaultAddr; // 0x0000000000000000000000000000000000000000
38 }
```

Explain contract

AI copilot

0 Listen on all transactions Filter with transaction hash or address

logs: 0 hash: 0x811...f421b

Tutorial 4.

LEARNETH

Tutorials list

Syllabus

4. Variables

4 / 19

when the current block was generated and `msg.sender` (line 15) to get the caller of the contract function's address.

A list of all Global Variables is available in the [Solidity documentation](#).

Watch video tutorials on [State Variables](#), [Local Variables](#), and [Global Variables](#).

★ Assignment

- Create a new public state variable called `blockNumber`.
- Inside the function `doSomething()`, assign the value of the current block number to the state variable `blockNumber`.

Tip: Look into the global variables section of the Solidity documentation to find out how to read the current block number.

Check Answer

Show answer

Next

Well done! No errors.

Compiled

introduction.sol

basicSyntax

LearnEthereum is modifying 4. Variables/variables_test.sol

```
4 contract Variables {
5   // State variables are stored on the blockchain.
6   string public text = "Hello";
7   uint public num = 123;
8   uint public blockNumber;
9
10  function doSomething() public { 22334 gas
11    // Local variables are not saved to the blockchain.
12    uint i = 456;
13
14    // Here are some global variables
15    uint timestamp = block.timestamp; // Current block timestamp
16    address sender = msg.sender; // address of the caller
17    blockNumber = block.number;
18  }
19 }
```

Explain contract

0 Listen on all transactions Filter with transaction hash or address

logs: 0 hash: 0xa11...f421b

Tutorial 5.

LEARNETH

Tutorials list

Syllabus

5.1 Functions - Reading and Writing to a State Variable

5 / 19

You can then set the visibility of a function and declare them `view` or `pure`, as we do for the `get` function if they don't modify the state. Our `get` function also returns values, so we have to specify the return types. In this case, it's a `uint` since the state variable `num` that the function returns is a `uint`.

We will explore the particularities of Solidity functions in more detail in the following sections.

Watch a video tutorial on [Functions](#).

★ Assignment

- Create a public state variable called `b` that is of type `bool` and initialize it to `true`.
- Create a public function called `get_b` that returns the value of `b`.

Check Answer

Show answer

Next

Well done! No errors.

Compiled

cSyntax.sol

primitiveDataTypes

LearnEthereum is modifying 5.1 Functions - Reading and Writing to a State Variable/readAndWrite_test.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract SimpleStorage {
5   // State variable to store a number
6   uint public num;
7   bool public b = true;
8   function get_b() public view returns(bool){ 2539 gas
9     return b;
10  }
11
12  // You need to send a transaction to write to a state variable.
13  function set(uint _num) public { 22536 gas
14    num = _num;
15  }
16
17  // You can read from a state variable without sending a transaction.
18  function get() public view returns (uint) { 2475 gas
19    return num;
20  }
21 }
```

Explain contract

0 Listen on all transactions Filter with transaction hash or address

logs: 0 hash: 0xa11...f421b

Tutorial 6.

LEARNETH

Tutorials list

Syllabus

5.2 Functions - View and Pure

6 / 19

You can declare a pure function using the keyword `pure`. In this contract, `add` (line 13) is a pure function. This function takes the parameters `i` and `j`, and returns the sum of them. It neither reads nor modifies the state variable `x`.

In Solidity development, you need to optimise your code for saving computation cost (gas cost). Declaring functions `view` and `pure` can save gas cost and make the code more readable and easier to maintain. Pure functions don't have any side effects and will always return the same result if you pass the same arguments.

[Watch a video tutorial on View and Pure Functions.](#)

★ Assignment

Create a function called `addToX2` that takes the parameter `y` and updates the state variable `x` with the sum of the parameter and the state variable `x`.

Check Answer

Show answer

Next

Well done! No errors.

Compiled

liveDataTypes.sol

variables.sc

Create a DApp with your contracts

Start now

LearnEth is modifying 5.2 Functions - View and Pure/viewAndPure_test.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract ViewAndPure {
5     uint public x = 1;
6
7     function addToX(uint y) public {
8         x = x + y;
9     }
10    // Promise not to modify the state.
11    function addToX(uint y) public view returns (uint) {
12        return x + y;
13    }
14
15    // Promise not to modify or read from the state.
16    function add(uint i, uint j) public pure returns (uint) {
17        return i + j;
18    }
19 }
```

Explain contract

AI copilot

0 Listen on all transactions

Filter with transaction hash or address

logs: 0 hash: 0x211...4210

Tutorial 7.

LEARNETH

Tutorials list

Syllabus

5.3 Functions - Modifiers and Constructors

7 / 19

contract (line 11) sets the initial value of the owner variable upon the creation of the contract.

[Watch a video tutorial on Function Modifiers.](#)

★ Assignment

- Create a new function, `increaseX` in the contract. The function should take an input parameter of type `uint` and increase the value of the variable `x` by the value of the input parameter.
- Make sure that `x` can only be increased.
- The body of the function `increaseX` should be empty.

Tip: Use modifiers.

Check Answer

Show answer

Next

Well done! No errors.

Compiled

3

readAndWrite.sol

viewAndPure

Create a DApp with your contracts

Start now

LearnEth is modifying 5.3 Function: Modifiers and Constructors/modifiersAndConstructo

```
51
52     if (i > 1) {
53         decrement(i - 1);
54     }
55 }
56 modifier increaseX2(uint value) {
57     x = x + value;
58     _;
59 }
60
61 // Function body is empty as required
62 function increaseX(uint value) public increaseX2(value) {
63     //
64 }
```

Explain contract

AI copilot

0 Listen on all transactions

Filter with transaction hash or address

logs: 0 hash: 0x211...4210

Tutorial 8.

LEARNETH

Tutorials list

Syllabus

5.4 Functions - Inputs and Outputs
8 / 19

Arrays can be used as parameters, as shown in the function `arrayInput` (line 71). Arrays can also be used as return parameters as shown in the function `arrayOutput` (line 76).

You have to be cautious with arrays of arbitrary size because of their gas consumption. While a function using very large arrays as inputs might fail when the gas costs are too high, a function using a smaller array might still be able to execute.

Watch a video tutorial on Function Outputs.

★ Assignment

Create a new function called `returnTwo` that returns the values `-2` and `true` without using a return statement.

Check Answer

Show answer

Next

Well done! No errors.

Compiled

viewAndPure.sol

LearnEth is modifying 5.4 Functions - Inputs and Outputs/inputsAndOutputs_test.sol

Create a DApp with your contracts

Start now

70 // Can use array for input

71 function arrayInput(uint[] memory _arr) public {} infinite gas

72

73 // Can use array for output

74 uint[] public arr;

75

76 function arrayOutput() public view returns (uint[] memory) { infinite gas

77 return arr;

78 }

79 function returnTwo() public 472 gas

80 pure

81 returns (int x, bool b){

82 x = -2;

83 b = true;

84 }

85 }

Explain contract

AI copilot

0 Listen on all transactions

Filter with transaction hash or address

logs: 0 hash: 0x411...7421b

Tutorial 9.

LEARNETH

Tutorials list

Syllabus

6. Visibility
9 / 19

When you uncomment the `testPrivateFunc` (lines 58-60) you get an error because the child contract doesn't have access to the private function `privateFunc` from the `Base` contract.

If you compile and deploy the two contracts, you will not be able to call the functions `privateFunc` and `internalFunc` directly. You will only be able to call them via `testPrivateFunc` and `testInternalFunc`.

Watch a video tutorial on Visibility.

★ Assignment

Create a new function in the `Child` contract called `testInternalVar` that returns the values of all state variables from the `Base` contract that are possible to return.

Check Answer

Show answer

Next

Well done! No errors.

Compiled

modifiersAndConstructor

LearnEth is modifying 6. Visibility/visibility_test.sol

Create a DApp with your contracts

Start now

58 // function testPrivateFunc() public {

59 // return privateFunc();

60 // }

61

62 // Internal function call be called inside child contracts.

63 function testInternalFunc() public pure override returns (string memory) {

64 return internalFunc();

65 }

66 function testInternalVar() infinite gas

67 public

68 view

69 returns (string memory, string memory)

70 {

71 return (internalVar, publicVar);

72 }

73 }

Explain contract

AI copilot

0 Listen on all transactions

Filter with transaction hash or address

logs: 0 hash: 0x411...7421b

Tutorial 10.

LEARNETH

Tutorials list

Syllabus

7.1 Control Flow - If/Else

10 / 19

If the first condition (line 6) of the foo function is not met, but the condition of the `else if` statement (line 8) becomes true, the function returns `1`.

Watch a video tutorial on the If/Else statement.

★ Assignment

Create a new function called `evenCheck` in the `IfElse` contract:

- That takes in a `uint` as an argument.
- The function returns `true` if the argument is even, and `false` if the argument is odd.
- Use a ternary operator to return the result of the `evenCheck` function.

Tip: The modulo (%) operator produces the remainder of an integer division.

Check Answer

Show answer

Next

Well done! No errors.

Compiled

ifiersAndConstructors.sol

input

Create a DApp with your contracts

Start now

LearnEth is modifying 7.1 Control Flow - If/Else/ifElse_test.sol

```
15 function ternary(uint _x) public pure returns (bool) {
16     // if (_x < 10) {
17     //     return 1;
18     // }
19     // return 2;
20
21     // shorthand way to write if / else statement
22     return _x < 10 ? 1 : 2;
23 }
24
25 function evenCheck(uint x) public returns (bool) {
26     if(x % 2 == 0){
27         return true;
28     }else{
29         return false;
30     }
31 }
32 }
```

Explain contract

AI copilot

0

Listen on all transactions

Filter with transaction hash or address

logs: 0 hash: 0x11...7421b

Tutorial 11.

LEARNETH

Tutorials list

Syllabus

7.2 Control Flow - Loops

11 / 19

prevent the second if statement (line 12) from being executed.

break

The `break` statement is used to exit a loop. In this contract, the break statement (line 14) will cause the for loop to be terminated after the sixth iteration.

Watch a video tutorial on Loop statements.

★ Assignment

- Create a public `uint` state variable called count in the `Loop` contract.
- At the end of the for loop, increment the count variable by 1.
- Try to get the count variable to be equal to 9, but make sure you don't edit the `break` statement.

Check Answer

Show answer

Next

Well done! No errors.

Compiled

sol

inputsAndOutputs.sol

visibility.sol

ifElse.sol 1

loops.sol

Create a DApp with your contracts

Start now

```
9 // for loop
10 for (uint i = 0; i < 10; i++) {
11     if (i == 3) {
12         // Skip to next iteration with continue
13         continue;
14     }
15     if (i == 5) {
16         // Exit loop with break
17         break;
18     }
19     count++;
20 }
21
22 // while loop
23 uint j;
24 while (j < 5) {
25     j++;
26     count++;
27 }
```

Explain contract

AI copilot

0

Listen on all transactions

Filter with transaction hash or address

logs: 0 hash: 0x11...7421b

Tutorial 12.

LEARNETH

Tutorials list

Syllabus

8.1 Data Structures - Arrays

12 / 19

element (line 46), or use a mapping. A mapping might be a better choice if we plan to remove elements in our data structure.

Array length

Using the length member, we can read the number of elements that are stored in an array (line 35).

[Watch a video tutorial on Arrays.](#)

★ Assignment

1. Initialize a public fixed-sized array called `arr3` with the values 0, 1, 2. Make the size as small as possible.
2. Change the `getArr()` function to return the value of `arr3`.

Check Answer

Show answer

Next

Well done! No errors.

Compiled

AndOutputs.sol

visibility.sol

ifElse.sol 1

loops.sol

ai

Create a DApp with your contracts

Start now

```
6    uint[] public arr;
7    uint[] public arr2 = [1, 2, 3];
8    uint[3] public arr3 = [0,1,2];
9
10   // Fixed sized array, all elements initialize to 0
11   uint[10] public myFixedSizeArr;
12
13   function get(uint i) public view returns (uint) { Infinite gas
14       return arr[i];
15   }
16
17   // Solidity can return the entire array.
18   // But this function should be avoided for
19   // arrays that can grow indefinitely in length.
20   function getArr() public view returns (uint[3] memory) { Infinite gas
21       return arr3;
22   }
23
24   function push(uint i) public { 46820 gas
```

Explain contract

AI c

0 Listen on all transactions

Filter with transaction hash or address

logs: 0 hash: 0xa11...7421b

Tutorial 13.

LEARNETH

Tutorials list

Syllabus

8.2 Data Structures - Mappings

13 / 19

brackets and assigning it a new value (line 16).

Removing values

We can use the delete operator to delete a value associated with a key, which will set it to the default value of 0. As we have seen in the arrays section.

[Watch a video tutorial on Mappings.](#)

★ Assignment

1. Create a public mapping `balances` that associates the key type `address` with the value type `uint`.
2. Change the functions `get` and `remove` to work with the mapping `balances`.
3. Change the function `set` to create a new entry to the `balances` mapping, where the key is the address of the parameter and the value is the balance associated with the address of the parameter.

Check Answer

Show answer

Next

Well done! No errors.

Compiled

Home

introduction.sol

mappings.sol X

Create a DApp with your contracts

Start now

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Mapping {
5     // Public mapping from address to uint
6     mapping(address => uint) public balances;
7
8     function get(address _addr) public view returns (uint) { 2872 gas
9         return balance
10     }
11     address internal _addr
12
13     function set(address _addr) public { 8.2 Data Structures - Mappings/mappings.sol 11:17
14         // Reads the n
15         balances[_addr] = _addr.balance;
16     }
17
18     function remove(address _addr) public { 5566 gas
19         delete balances[_addr];
20     }
21 }
```

Tutorial 14.

8.3 Data Structures - Structs

14 / 19

key-value mapping: We provide the name of the struct and the keys and values as a mapping inside curly braces (line 19).

initialize and update a struct: We initialize an empty struct first and then update its member by assigning it a new value (line 23).

Accessing structs

to access a member of a struct we can use the dot operator (line 33).

Updating structs

to update a structs' member we also use the dot operator and assign it a new value (lines 39 and 45).

[Watch a video tutorial on Structs.](#)

★ Assignment

Create a function `remove` that takes a `uint` as a parameter and deletes a struct member with the given index in the `todos` mapping.

Check Answer

Show answer

Next

Well done! No errors.

```
30 // you don't actually need this function.
31 function get(uint _index) public view returns (string memory text, bool completed) {
32     Todo storage todo = todos[_index];
33     return (todo.text, todo.completed);
34 }
35
36 // update text
37 function update(uint _index, string memory _text) public {
38     Todo storage todo = todos[_index];
39     todo.text = _text;
40 }
41
42 // update completed
43 function toggleCompleted(uint _index) public {
44     Todo storage todo = todos[_index];
45     todo.completed = !todo.completed;
46 }
47 function remove(uint _index) public {
48     delete todos[_index];
49 }
50 }
```

Tutorial 15.

8.4 Data Structures - Enums

15 / 19

Updating an enum value

We can update the enum value of a variable by assigning it the `uint` representing the enum member (line 30). Shipped would be 1 in this example. Another way to update the value is using the dot operator by providing the name of the enum and its member (line 35).

Removing an enum value

We can use the delete operator to delete the enum value of the variable, which means as for arrays and mappings, to set the default value to 0.

[Watch a video tutorial on Enums.](#)

★ Assignment

1. Define an enum type called `Size` with the members `S`, `M`, and `L`.
2. Initialize the variable `sizes` of the enum type `Size`.
3. Create a getter function `getSize()` that returns the value of the variable `sizes`.

Check Answer

Show answer

Next

Well done! No errors.

```
20 // Shipped - 1
21 // Accepted - 2
22 // Rejected - 3
23 // Canceled - 4
24 function get() public view returns (Status) {
25     return status;
26 }
27
28 // Update status by passing uint into input
29 function set(Status _status) public {
30     status = _status;
31 }
32
33 // You can update to a specific enum like this
34 function cancel() public {
35     status = Status.Canceled;
36 }
37
38 // delete resets the enum to its first value, 0
39 function reset() public {
40     delete status;
41 }
42 enum Size {
43     S,M,L
44 }
45 Size public sizes;
46 function getSize() public view returns (Size){
47     return sizes;
48 }
```

Tutorial 16.

9. Data Locations
16 / 19

As we said in the beginning, when creating contracts we have to be mindful of gas costs. Therefore, we need to use data locations that require the lowest amount of gas possible.

★ Assignment

1. Change the value of the `myStruct` member `foo`, inside the `function f`, to 4.
2. Create a new struct `myMemStruct2` with the data location `memory` inside the `function f` and assign it the value of `myMemStruct`. Change the value of the `myMemStruct2` member `foo` to 1.
3. Create a new struct `myMemStruct3` with the data location `memory` inside the `function f` and assign it the value of `myStruct`. Change the value of the `myMemStruct3` member `foo` to 3.
4. Let the function `f` return `myStruct`, `myMemStruct2`, and `myMemStruct3`.

Tip: Make sure to create the correct return types for the function `f`.

Check AnswerShow answer

Next

Well done! No errors.

26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53

```
MyStruct memory myMemStruct2 = myMemStruct;  
myMemStruct2.foo = 1;  
  
// 3. Create myMemStruct3 in memory, assign myStruct, change foo to 3  
MyStruct memory myMemStruct3 = myStruct;  
myMemStruct3.foo = 3;  
  
// 4. Return myStruct, myMemStruct2, and myMemStruct3  
return (myStruct, myMemStruct2, myMemStruct3);  
}  
  
function _f( /* undefined gas  
uint[] storage _arr,  
mapping(uint => address) storage _map,  
MyStruct storage _myStruct  
) internal {  
    // do something with storage variables  
}  
  
// You can return memory variables  
function g(uint[] memory _arr) public returns (uint[] memory) { /* infinite gas  
    // do something with memory array  
}  
  
function h(uint[] calldata _arr) external { /* 468 gas  
    // do something with calldata array  
}
```

Tutorial 17.

10.1 Transactions - Ether and Wei
17 / 19

`wei`
Wei is the smallest subunit of *Ether*, named after the cryptographer *Wei Dai*. *Ether* numbers without a suffix are treated as `wei` (line 7).

`gwei`
One `gwei` (giga-wei) is equal to 1,000,000,000 (10^9) `wei`.

`ether`
One `ether` is equal to 1,000,000,000,000,000,000 (10^{18}) `wei` (line 11).
Watch a video tutorial on Ether and Wei.

★ Assignment

1. Create a `public uint` called `oneGwei` and set it to 1 `gwei`.
2. Create a `public bool` called `isOneGwei` and set it to the result of a comparison operation between 1 `gwei` and 10^9 .

Tip: Look at how this is written for `gwei` and `ether` in the contract.

Check AnswerShow answer

Next

Well done! No errors.

1
2
3
4
5
6
7
8
9
10
11
12
13

```
// SPDX-License-Identifier: MIT  
pragma solidity ^0.8.3;  
  
contract EtherUnits {  
    uint public oneWei = 1 wei;  
    // 1 wei is equal to 1  
    bool public isOneWei = 1 wei == 1;  
    uint public oneGwei = 1 gwei;  
    bool public isOneGwei = 1 gwei == 1e9;  
    uint public oneEther = 1 ether;  
    // 1 ether is equal to 10^18 wei  
    bool public isOneEther = 1 ether == 1e18;  
}
```

Tutorial 18.

10.2 Transactions - Gas and Gas Price
18 / 19

Gas prices are denoted in gwei.

Gas limit

When sending a transaction, the sender specifies the maximum amount of gas that they are willing to pay for. If they set the limit too low, their transaction can run out of gas before being completed, reverting any changes being made. In this case, the gas was consumed and can't be refunded.

Learn more about gas on ethereum.org.

Watch a video tutorial on Gas and Gas Price.

★ Assignment

Create a new `public` state variable in the `Gas` contract called `cost` of the type `uint`. Store the value of the gas cost for deploying the contract in the new variable, including the cost for the value you are storing.

Tip: You can check in the Remix terminal the details of a transaction, including the gas cost. You can also use the Remix plugin *Gas Profiler* to check for the gas cost of transactions.

Check AnswerShow answer

Next

Well done! No errors.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Gas {
5     uint public i = 0;
6     uint public cost = 170367;
7
8     // Using up all of the gas that you send causes your transaction to fail
9     // State changes are undone.
10    // Gas spent are not refunded.
11    function forever() public {
12        // Here we run a loop until all of the gas are spent
13        // and the transaction fails
14        while (true) {
15            i += 1;
16        }
17    }
18 }
```

Explain contract

0 Listen on all transactions Filter with transaction hash or address

Type the library name to see available commands.

Tutorial 19.

10.3 Transactions - Sending Ether
19 / 19

If you change the parameter type for the functions `sendViaTransfer` and `sendViaSend` (line 33 and 38) from `payable address` to `address`, you won't be able to use `transfer()` (line 35) or `send()` (line 41).

Watch a video tutorial on Sending Ether.

★ Assignment

Build a charity contract that receives Ether that can be withdrawn by a beneficiary.

- Create a contract called `Charity`.
- Add a public state variable called `owner` of the type `address`.
- Create a donate function that is public and payable without any parameters or function code.
- Create a withdraw function that is public and sends the total balance of the contract to the `owner` address.

Tip: Test your contract by deploying it from one account and then sending Ether to it from another account. Then execute the withdraw function.

Check AnswerShow answer

Next

Well done! No errors.

```
1 pragma solidity ^0.8.3;
2
3 contract Charity {
4     // 1. Owner address stored as a public state variable
5     address public owner;
6
7     // Set deployer as the owner in the constructor
8     constructor() {
9         // 163845 gas 139400 gas
10        owner = msg.sender;
11    }
12
13    // 2. Donate function: public and payable, no parameters or code body
14    function donate() public payable {}
15
16    // 3. Withdraw function: sends the total contract balance to the owner
17    function withdraw() public {
18        // infinite gas
19        // Send entire contract balance to the owner using .call
20        (bool sent, ) = payable(owner).call(value: address(this).balance, "");
21        require(sent, "Failed to send Ether");
22    }
23 }
```

Explain contract

0 Listen on all transactions Filter with transaction hash or address

Type the library name to see available commands.

Conclusion

Through this experiment, the fundamentals of Solidity programming were explored by completing practical assignments in the Remix IDE. Concepts such as data types, variables, functions, visibility, modifiers, constructors, control flow, data structures, and transactions were implemented and understood. The hands-on practice helped in designing, compiling, and deploying smart contracts on the Remix VM, thereby strengthening the understanding of blockchain concepts. This experiment provided a strong foundation for developing and managing smart contracts efficiently.